



Container to WASM converter

[ktock.github.io/container2wasm-demo/](#)

Apache-2.0 license

Code of conduct

2.2k stars

96 forks

21 watching

Branches

Activity

Custom properties

Tags

Public repository

main

12 Branches

20 Tags

Go to file

Go to file

+

Add file

Code

ktock

Merge pull request #440 from container2wasm/dependabot/go_modules/git...

035516f · 3 days ago

.github	test golangci-lint v2	2 weeks ago
cmd	test golangci-lint v2	2 weeks ago
config	Enable QEMU Wasm when --to-js is specified	3 months ago
docs/images	Enable QEMU Wasm when --to-js is specified	3 months ago
examples	Enable QEMU Wasm when --to-js is specified	3 months ago
extras	bump github.com/containers/gvisor-tap-vsoc...	2 weeks ago
tests	bump github.com/containers/gvisor-tap-vsoc...	2 weeks ago
version	first commit	2 years ago
.gitignore	first commit	2 years ago
.golangci.yml	test golangci-lint v2	2 weeks ago
CODE_OF_CONDUCT.md	Add more docs about project governance	8 months ago
CONTRIBUTING.md	Add docs about project governance	8 months ago
Dockerfile	Merge pull request #402 from container2was...	2 months ago
Dockerfile.test	Merge pull request #438 from container2was...	5 days ago
GOVERNANCE.md	docs: start working on governance docs.	2 months ago
LICENSE	first commit	2 years ago
MAINTAINERS	Add docs about project governance	8 months ago
Makefile	Makefile: add make vendor	3 weeks ago
README.md	Enable QEMU Wasm when --to-js is specified	3 months ago
ROADMAP.md	Add more docs about project governance	8 months ago
embed.go	first commit	2 years ago
go.mod	build(deps): bump github.com/moby/sys/user ...	4 days ago
go.sum	build(deps): bump github.com/moby/sys/user ...	4 days ago

container2wasm: Container to WASM converter

container2wasm is a container-to-wasm image converter that enables to run the container on WASM.

- Converts a container to WASM with emulation by Bochs (for x86_64 containers), TinyEMU (for riscv64 containers) and QEMU.
- Runs on WASI runtimes (e.g. wasmtime, wamr, wasmer, wasmedge, wazero)
- Runs on browser
- x86_64, riscv64 or AArch64 containers are recommended.

This is an experimental software.

Demo page of containers on browser (debian,python,node,vim): <https://ktock.github.io/container2wasm-demo/>

Examples

Container Image to WASM (WASI)

```
$ c2w ubuntu:22.04 out.wasm
```



The above command converts `ubuntu:22.04` container image to WASI image (`out.wasm`).

NOTE1: For selecting the container image's architecture other than `amd64` , use `--target-arch` flag of `c2w` (e.g. `c2w --target-arch=riscv64 riscv64/ubuntu:22.04 out.wasm`).

NOTE2: `x86_64` or `riscv64` container is recommended. Other platform's containers should work but slow because of additional emulation.

The generated image runs on WASI runtimes:

```
$ wasmtime out.wasm uname -a
Linux localhost 6.1.0 #1 PREEMPT_DYNAMIC Mon Jun  5 11:57:09 UTC 2023 x86_64 x86_64 x86_64 GNU/Linux
$ wasmtime out.wasm ls /
bin  dev  home lib32 libx32 mnt  proc run  srv  tmp  var
boot etc  lib  lib64 media  opt  root sbin sys  usr
```



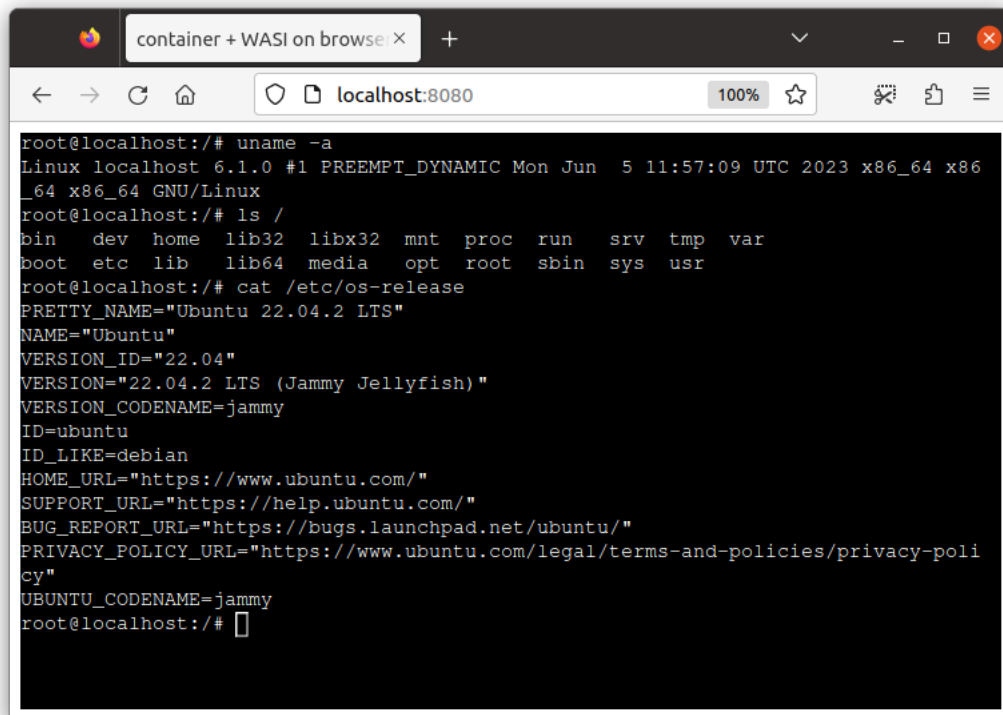
Directory mapped from the host is accessible on the container.

```
$ mkdir -p /tmp/share/ && echo hi > /tmp/share/from-host
$ wasmtime --mapdir /mnt/share::/tmp/share out.wasm cat /mnt/share/from-host
hi
```



Please refer to [./examples/networking/wasi/](https://ktock.github.io/container2wasm-examples/networking/wasi/) for enabling networking

Container on Browser



```
root@localhost:/# uname -a
Linux localhost 6.1.0 #1 PREEMPT_DYNAMIC Mon Jun  5 11:57:09 UTC 2023 x86_64 x86_64 x86_64 GNU/Linux
root@localhost:/# ls /
bin  dev  home  lib32  libx32  mnt  proc  run  srv  tmp  var
boot  etc  lib  lib64  media  opt  root  sbin  sys  usr
root@localhost:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.2 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.2 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
root@localhost:/#
```

You can run the container on browser as well. There are two methods for running the container on browser.

WASI on browser

This example converts the container to WASI and runs it on browser.

Please also refer to [./examples/wasi-browser](#) for details.

The following command generates a WASI image.

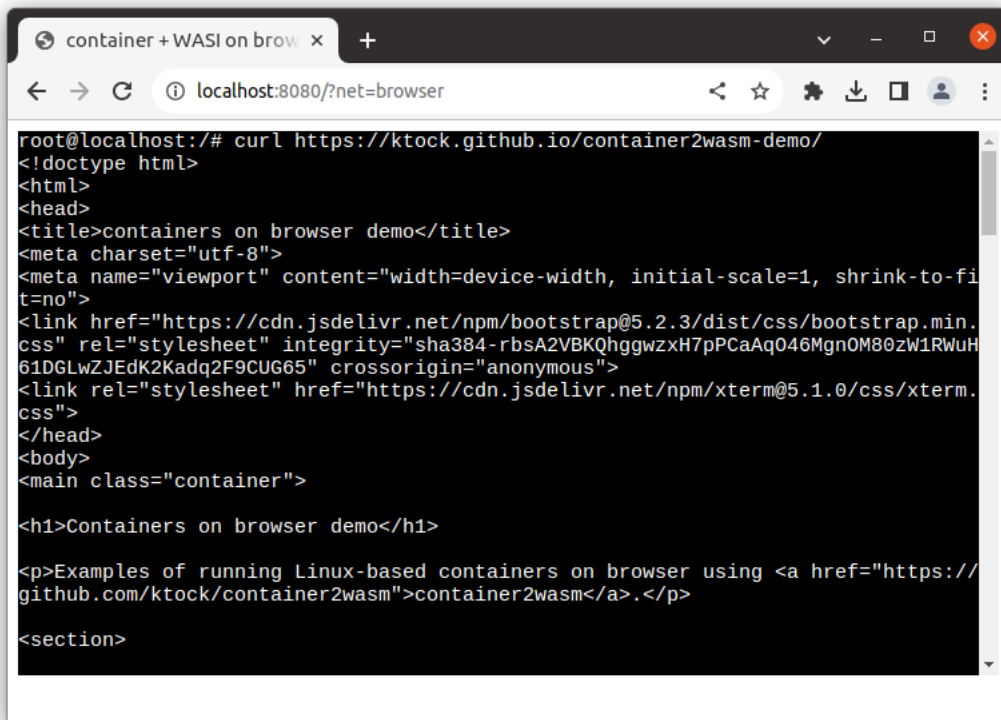
```
$ c2w ubuntu:22.04 /tmp/out-js2/htdocs/out.wasm
```

The following is an example of running the image on browser relying on [xterm-pty](#) and [browser_wasi_shim](#). This example serves the image on `localhost:8080` using apache http server.

```
$ cp -R ./examples/wasi-browser/* /tmp/out-js2/ && chmod 755 /tmp/out-js2/htdocs
$ docker run --rm -p 8080:80 \
  -v "/tmp/out-js2/htdocs:/usr/local/apache2/htdocs:ro" \
  -v "/tmp/out-js2/xterm-pty.conf:/usr/local/apache2/conf/extra/xterm-pty.conf:ro" \
  --entrypoint=/bin/sh httpd -c 'echo "Include conf/extra/xterm-pty.conf" >>
  /usr/local/apache2/conf/httpd.conf && httpd-foreground'
```

You can run the container on browser via `localhost:8080`.

WASI on browser with networking



Container can also perform networking. This section is the demo of using curl command in the container.

Tested only on Chrome. The example might not work on other browsers.

```
$ cat <<EOF | docker build -t debian-curl -
FROM debian:sid-slim
RUN apt-get update && apt-get install -y curl
EOF
$ c2w debian-curl /tmp/out-js2/htdocs/out.wasm
```



This example serves the image on `localhost:8080` using apache http server. The following also puts the [network stack runnable on browser](#) to the document root.

```
$ cp -R ./examples/wasi-browser/* /tmp/out-js2/ && chmod 755 /tmp/out-js2/htdocs
$ wget -O /tmp/out-js2/htdocs/c2w-net-proxy.wasm
https://github.com/ktock/container2wasm/releases/download/v0.5.0/c2w-net-proxy.wasm
$ docker run --rm -p 8080:80 \
  -v "/tmp/out-js2/htdocs:/usr/local/apache2/htdocs/:ro" \
  -v "/tmp/out-js2/xterm-pty.conf:/usr/local/apache2/conf/extra/xterm-pty.conf:ro" \
  --entrypoint=/bin/sh httpd -c 'echo "Include conf/extra/xterm-pty.conf" >>
/usr/local/apache2/conf/httpd.conf && httpd-foreground'
```



You can run the container on browser with several types of configurations:

- `localhost:8080/?net=browser` : Container with networking. [Network stack c2w-net-proxy](#) implemented based on [gvisor-tap-vsock](#) runs on browser and forwards HTTP/HTTPS packets using the browser's Fetch API. The set of accessible sites is restricted by the browser configuration (e.g. CORS restriction). See also [./examples/networking/fetch](#) for details.
- `localhost:8080/?net=delegate=ws://localhost:8888` : Container with networking. You need to run [user-space network stack c2w-net](#) implemented based on [gvisor-tap-vsock](#) on the host (outside of browser). It forwards all packets received from the browser over WebSocket. See also [./examples/networking/websocket](#) for details and configuration. (tested only on Linux)
- `localhost:8080` : Container without networking.

Please also refer to [./examples/networking/](#) for details about networking.

emscripten on browser

This example uses emscripten for converting the container to WASM. This leverages [QEMU Wasm](#) which is QEMU ported to browser with JIT compilation (TCG) enabled. Also refer to [./examples/emscripten/](#) for details.

The following command generates a WASM image and a JS file runnable on browser.

```
$ c2w --to-js alpine:3.20 /tmp/out-js/htdocs/
```

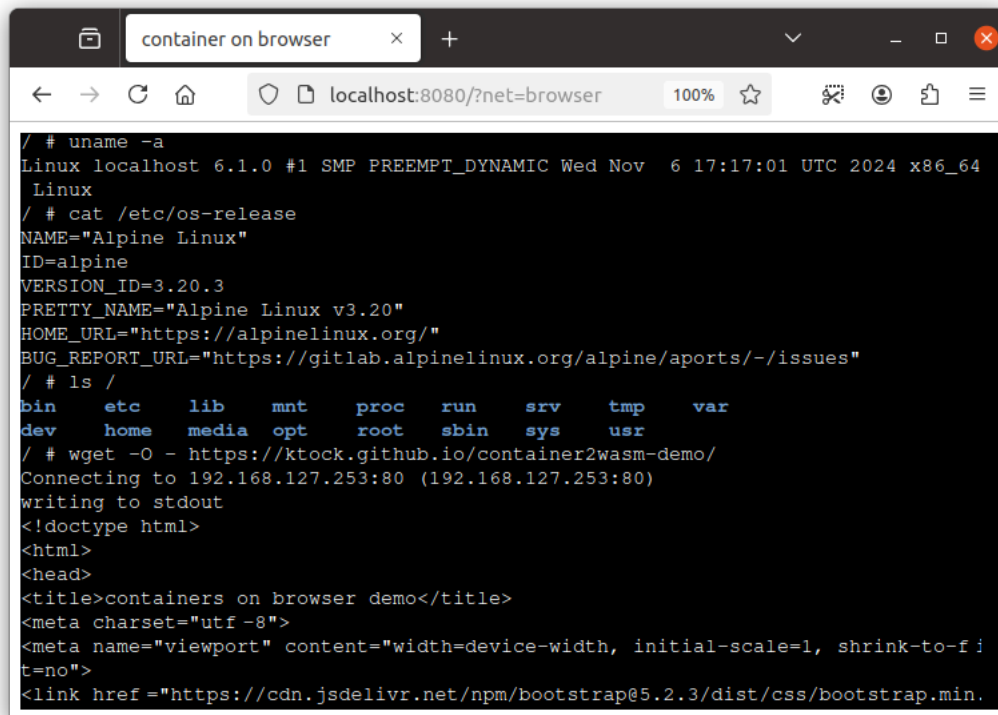


The following is an example of running the image on browser relying on [xterm-pty](#). This example serves the image on localhost:8080 using apache http server.

```
$ ( cd ./examples/emscripten/htdocs/ && npx webpack && cp -R index.html dist vendor/xterm.css /tmp/out-js4/htdocs/ )
$ wget -O /tmp/c2w-net-proxy.wasm https://github.com/ktock/container2wasm/releases/download/v0.5.0/c2w-net-proxy.wasm
$ cat /tmp/c2w-net-proxy.wasm | gzip > /tmp/out-js4/htdocs/c2w-net-proxy.wasm.gzip
$ cp ./examples/emscripten/xterm-pty.conf /tmp/out-js4/
$ docker run --rm -p 8080:80 \
  -v "/tmp/out-js4/htdocs:/usr/local/apache2/htdocs/:ro" \
  -v "/tmp/out-js4/xterm-pty.conf:/usr/local/apache2/conf/extra/xterm-pty.conf:ro" \
  --entrypoint=/bin/sh httpd -c 'echo "Include conf/extra/xterm-pty.conf" >> /usr/local/apache2/conf/httpd.conf'
```

You can run the container on browser with several types of configurations:

- localhost:8080/?net=browser : Container with networking. [Network stack](#) [c2w-net-proxy](#) implemented based on [gvisor-tap-vsock](#) runs on browser and forwards HTTP/HTTPS packets using the browser's Fetch API. The set of accessible sites is restricted by the browser configuration (e.g. CORS restriction). See also [./examples/emscripten/](#) for details.
- localhost:8080/?net=delegate=ws://localhost:8888 : Container with networking. You need to run [user-space network stack](#) [c2w-net](#) implemented based on [gvisor-tap-vsock](#) on the host (outside of browser). It forwards all packets received from the browser over WebSocket. See also [./examples/emscripten/](#) for details and configuration. (tested only on Linux)
- localhost:8080 : Container without networking.



[README](#) [Code of conduct](#) [Apache-2.0 license](#)



Please also refer to [./examples/networking/](#) for details about networking.

Getting Started

- requirements
 - Docker 18.09+ (w/ DOCKER_BUILDKIT=1)
 - [Docker Buildx](#) v0.8+ (recommended) or docker build (w/ DOCKER_BUILDKIT=1)

You can install the converter command `c2w` using one of the following methods.

NOTE: The output binary also contains [c2w-net](#) which a command usable for controlling networking feature (please see also [./examples/networking](#) for details).

Release binaries

Binaries are available from <https://github.com/ktock/container2wasm/releases> Extract the tarball and put the binary somewhere under \$PATH .

Building binaries using make

Go 1.19+ is needed.

```
make
sudo make install
```



Command reference

c2w

Converts a container image into a WASM image and writes it to the specified path (default: `out.wasm` at the current directory).

Usage: `c2w [options] image-name [output file]`

- `image-name` : container image name (will be pulled from the registry if it doesn't exist in Docker)
- `[output file]` : path to the result WASM file.

Sub commands

- `help, h` : Shows a list of commands or help for one command

Options

- `--assets value` : Custom location of build assets.
- `--dockerfile value` : Custom location of Dockerfile (default: embedded to this command)
- `--builder value` : Builder command to use (default: "docker")
- `--target-arch value` : target architecture of the source image to use (default: "amd64")
- `--build-arg value` : Additional build arguments (please see Dockerfile for available build args)
- `--to-js` : convert the container to WASM using emscripten
- `--debug-image` : Enable debug print in the output image
- `--show-dockerfile` : Show default Dockerfile
- `--legacy` : Use "docker build" instead of buildx (no support for assets flag) (default:false)
- `--external-bundle` : Do not embed container image to the Wasm image but mount it during runtime
- `--help, -h` : show help
- `--version, -v` : print the version

Run-time flags for WASM image

You can specify run-time flags to the generated wasm image for configuring the execution (e.g. for changing command to run in the container).

Usage: `out.wasm [options] [COMMAND] [ARG...]`

- `[COMMAND] [ARG...]` : command to run in the container. (default: commands specified in the image config)

Options

- `-entrypoint <command>` : entrypoint command. (default: entrypoint specified in the image config)
- `-no-stdin` : disable stdin. (default: false)

Example:

The following changes the container's entrypoint to `echo` and pass `hello` to the arguments.

```
wasmtime -- /app/out.wasm --entrypoint=echo hello
```



Directory mapping

Directory mapped from the host is accessible on the container.

```
$ mkdir -p /tmp/share/ && echo hi > /tmp/share/hi
$ wasmtime --mapdir /test/dir/share::/tmp/share /app/out.wasm ls /test/dir/share/
hi
```



Motivation

Though more and more programming languages start to support WASM, it's not easy to run the existing programs on WASM. This sometimes requires re-implementing and re-compiling them and costs extra time for development. This is a PoC converter tries to solve it by enabling running unmodified containers on WASM.

How does it work

containr2wasm creates a WASM image that runs the container and the Linux kernel on the emulated CPU.

The following shows the technical details:

- Builder: [BuildKit](#) runs the conversion steps written in Dockerfile.
- Emulator: [Bochs](#) emulates x86_64 CPU on WASM. [TinyEMU](#) emulates RISC-V CPU on WASM. For `--to-js` flag, QEMU ([QEMU Wasm](#)) is used. They're compiled to WASM using [wasi-sdk](#) (for WASI and on-browser) and [emscripten](#) (for on-browser).
- Guest OS: Linux runs on the emulated CPU. [runc](#) starts the container. Non-x86 and non-RISC-V containers runs with additional emulation by QEMU installed via [tonistiigi/binfmt](#).
- Directory Mapping: WASI filesystem API makes host directories visible to the emulator. Emulators mount them to the guest linux via `virtio-9p`.
- Packaging: [wasi-vfs](#) (for WASI and on-browser) and [emscripten](#) (for on-browser) are used for packaging the dependencies. The kernel is pre-booted during the build using [wizer](#) to minimize the startup latency (for WASI only as of now).
- Networking: Browser's Fetch API or WebSocket is used for on-browser image. `sock_*` API is used for WASI. [gvisor-tap-vsock](#) can be used as the networking stack. (docs: [./examples/networking/](#))
- Security: The converted container runs in the sandboxed WASM (WASI) VM with the limited access to the host system.

WASI Runtimes Integration Status

- ✓ : supported
- 🚧 : WIP
- NOTE: WASI features not listed here are untested (future version will support more features)

x86_64 containers

runtime	stdio	mapdir	networking	note
wasmtime	✓	✓	✓ (w/ host-side network stack)	
wamr(wasm-micro-runtime)	✓	✓	🚧	
wazero	✓	✓	✓ (w/ host-side network stack)	
wasmer	🚧 (stdin unsupported)	✓	🚧	non-blocking stdin doesn't seem to work
wasmedge	🚧 (stdin unsupported)	✓	🚧	non-blocking stdin doesn't seem to work

risc-v and other architecutre's containers

runtime	stdio	mapdir	networking	note
wasmtime	✓	✓	✓ (w/ host-side network stack)	
wamr(wasm-micro-runtime)	✓	✓	🚧	
wazero	✓	✓	✓ (w/ host-side network stack)	
wasmer	🚧 (stdin unsupported)	✓	🚧	non-blocking stdin doesn't seem to work

runtime	stdio	mapdir	networking	note
wasmedge	🚧 (stdin unsupported)	✓	🚧	non-blocking stdin doesn't seem to work

Similar projects

There are several container runtimes support running WASM applications, but they don't run containers on WASM.

- WASM on container runtimes
 - Docker+Wasm integration: <https://docs.docker.com/desktop/wasm/>
 - runwasi: <https://github.com/containerd/runwasi>
 - youki: <https://github.com/containers/youki>
 - crun: <https://github.com/containers/crun>
 - krustlet: <https://github.com/krustlet/krustlet>

There are emulators that support running linux on WASM, but they don't support WASI.

- x86 on WASM
 - v86: <https://github.com/copy/v86>
- RISCv on WASM
 - TinyEMU: <https://bellard.org/tinyemu/>

Releases 20

 **v0.8.0** Latest
on Jan 30

[+ 19 releases](#)

Packages

No packages published

Contributors 6



Languages

